Московский государственный университет имени М. В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра системного программирования

Денисьев Илья Денисович

Автоматизация тестирования драйвера FAT32 при помощи генерации корректных образов файловой системы с использованием её формальной спецификации на основе зависимых типов

Курсовая работа

Научный руководитель:
к.ф.-м.н. Хорошилов Алексей Владимирович
Научный консультант:
Буздалов Денис Викторович

Москва, 2025

Аннотация

Автоматизация тестирования драйвера FAT32 при помощи генерации корректных образов файловой системы с использованием её формальной спецификации на основе зависимых типов

Денисьев Илья Денисович

Файловые системы составляют одну из самых важных частей современных операционных систем, являясь большим источником уязвимостей для них. В связи с этим, задача верификации и тестирования соответствующих драйверов стоит особенно остро. В данной работе представлен метод генерации семантически корректных образов FAT32 с использованием формальной спецификации на основе зависимых типов. Разработанный генератор `fat32-model` написан на Idris 2 с использованием библиотеки DepTyCheck. С помощью данного инструмента было найдено несоответствие спецификации в операционных системах Windows, Linux и macOS. Кроме того, проведённый с помощью syzkaller анализ показал, что использование сгенерированных `fat32-model` образов в качестве начальных улучшает покрытие кода ядра Linux при фаззинге по сравнению с использованием образов, вшитых в syzkaller по умолчанию.

Abstract

Automation of FAT32 driver testing by means of correct filesystem image generation using its formal specification based on dependent types

Ilya Denisyev

Filesystems are one of the most important parts of contemporary operating systems, being a big source of vulnerabilities for them. In this regard, verification and testing of filesystem drivers is especially relevant. We present a method for generating semantically correct FAT32 images using a formal specification based on dependent types. The `fat32-model` generator was developed with Idris 2 using the DepTyCheck library. With this tool we were able to find that FAT32 implementations in Windows, Linux and macOS do not fully conform to the specification, Moreover, an analysis conducted with syzkaller has shown that using images, generated with `fat32-model`, while fuzzing increased Linux kernel code coverage, compared to using default images shipped with syzkaller.

Содержание

1 Введение **4**

2 Постановка задачи **6**

3 Обзор существующих решений **6**

4 Исследование и построение решения задачи **8**

4.1 Генерация образов 8

4.1.1 Основная структура (дерево ФС) 10

4.1.2 Содержимое файлов 11

4.1.3 Имена файлов и директорий 11

4.1.4 Отображение виртуальных номеров кластеров в физические 12

4.1.5 Метаинформация образа 13

4.1.6 Загрузочный сектор 13

4.1.7 Блок FSInfo 13

4.1.8 Сборка бинарного образа 13

4.2 Оценка эффективности влияния на фаззеры 13

5 Результаты **15**

5.1 Генератор 15

5.2 Несоответствие спецификации 16

5.3 Влияние на работу фаззера 17

5.3.1 Сложности работы с syzkaller'ом 17

5.3.2 Анализ покрытия 17

6 Заключение **19**

Список литературы **20**

1 Введение

Одной из главных функций операционных систем является работа с различными файловыми системами. Поскольку большинство современных ядер ОС* — монолитные, соответствующие драйверы являются потенциальными векторами атаки. Ошибки в драйверах ФС† могут привести к аварийным остановкам и к уязвимостям, потенциально компрометирующим всю систему. Всё это обеспечивает актуальность тщательного тестирования этих драйверов. Однако эта задача имеет свои сложности — во-первых, драйвер как правило приходится рассматривать в виде «чёрного ящика», имея возможность получать и изменять его состояние только посредством определённых системных вызовов. Во-вторых, валидность тех или иных действий с файловой системой зависит от её текущего состояния, которое ещё и изменяется в процессе тестирования. Сложность системы и отсутствие доступа к её полному внутреннему состоянию означает, что ручное написание тестов с полным предугадыванием результата трудоёмко и малоэффективно.

Одним из широко используемых способов тестирования ФС является *фаззинг* — автоматическая генерация случайных входных данных с целью спровоцировать ошибки или падения программы. При этом довольно часто генерация происходит за счёт мутации каких-то существующих (в том числе ранее сгенерированных) входных данных (в данном случае — изначальный образ ФС и последовательность действий с ней), возможно, с некой обратной связью (например, степень покрытия кода программы). Полученные данные вовсе не обязательно являются валидными, что может быть и желаемым эффектом (например, для проверки того, как программа справляется с некорректными данными и обработкой ошибок), но имеет и недостатки — потенциально меньшее покрытие более «глубоких» путей в коде и неприменимость таких данных для функционального тестирования (т.е. не только попытка вызвать падения, но и проверка корректности исполнения действий).

Для решения обозначенных проблем в данной работе представлен метод (далее называемый `fat32-model`) property-based‡ тестирования файловых систем (на примере FAT32), заключающийся в генерации семантически корректных образов ФС с использованием её формальной спецификации на основе зависимых типов. *Property-based* означает, что тестирование производится на случайно сгенерированных данных и что проверяются только определённые свойства тестируемой системы, выраженные как общие зависимости результата работы системы от заданных входных данных.

---

*операционных систем

†файловых систем

‡основанное на свойствах

Это противопоставляется проверке соответствия некоторому эталону (таким образом решается проблема предугадывания результата). Свойствами для property-based тестирования в данном случае являются:

1. отсутствие фатальных ошибок ядра ОС
2. успешное монтирование образа
3. корректная работа `ls`

Программа-генератор написана на *Idris 2* [1], [2], [3] — чистом функциональном языке программирования общего назначения с *зависимыми типами*, т.е. типами, которые могут зависеть от значений. Богатая система типов языка позволяет описывать сложные структуры данных вместе с различными инвариантами, которые должны для неё выполняться, а используемая библиотека *DepTyCheck* [4], [5] (разрабатываемая в ИСП РАН) позволяет *деривировать* (т.е. автоматически выводить код части программы на этапе компиляции с помощью механизма метапрограммирования *Elab* [6]) генераторы значений типа по его описанию. Полученный метод открывает возможности для дальнейшего функционального тестирования драйвера, а также позволяет использовать более разнообразные образы в качестве изначальных данных для мутации фаззерами, потенциально улучшая их эффективность.

2 Постановка задачи

Создать формальную спецификацию с использованием зависимых типов для семантически корректных образов файловой системы FAT32, соответствующих спецификации [7], а также на её основе разработать генератор таких образов. Провести тестирование некоторых существующих реализаций с использованием этого генератора. Исследовать влияние сгенерированных образов файловой системы на качество фаззинга, проводимое при помощи существующих инструментов.

3 Обзор существующих решений

Для фаззинга ядра Linux (и не только) широко применяется *syzkaller* [8] — coverage-guided[§] фаззер, который генерирует последовательности системных вызовов, постепенно мутируя их из начального корпуса, и запускает их в виртуальных машинах, собирая покрытие. Syzkaller применяется в том числе и для тестирования драйверов файловых систем, и в процессе его работы мутируются образы ФС (в качестве входных данных для псевдосистемного вызова `syz_mount_image`). Однако получаемые таким способом образы довольно часто получаются некорректными. Кроме того, изначальные образы для мутации получаются [9] с помощью `mkfs.fat` и обычного копирования файлов, что потенциально ограничивает пространство возможных образов (поскольку, например, при записи в него файлов драйвер обычно не будет случайно раскидывать кластеры файлов, а будет заполнять их последовательно).

Исследователями из технологического института Джорджии, пытавшимися увеличить эффективность фаззинга конкретно файловых систем, был разработан *Janus* [10]. Три основных преимущества перед syzkaller:

1. Образы мутируются с учётом их структуры, при надобности пересчитываются контрольные суммы и т.д. (таким образом, работа идёт по большей части с корректными образами)
2. Janus поддерживает предполагаемое состояние ФС в памяти в процессе фаззинга, что позволяет генерировать только корректные в данный момент действия
3. Вместо виртуальных машин используется LKL (Linux Kernel Library) для улучшения воспроизводимости падений

Однако Janus всё ещё не занимается генерацией образов с нуля, а мутирует заданные начальные образы, и если они недостаточно разнообразные (как в случае с syzkaller), то могут быть такие же проблемы с потенциальным ограничением пространства рассматриваемых образов.

[§]управляемый покрытием

Инструменты интерактивного доказательства теорем (в число которых входит и Idris) также не в первый раз используются для обеспечения качества драйверов файловых систем — существует формализация [11] FAT32 в системе ACL2. В рамках этой работы были также разработаны формально верифицированные реализации некоторых системных вызовов для FAT32, но не метод генерации образов, соответствующих модели. Тем не менее, рассматриваемая в статье методология инкрементального построения модели путём постепенного понижения уровня абстракции хорошо переносится и на Idris, и построение `fat32-model` происходило похожим образом.

4 Исследование и построение решения задачи

Для решения поставленной задачи требовалось разработать генератор, проверить соответствие получаемых образов заданным свойствам и оценить эффективность метода по различным метрикам:

1. покрытие кода ядра (в особенности именно кода драйвера FAT32)
2. количество найденных несоответствий спецификации
3. количество обнаруженных падений

Для этого сгенерированные образы используются как начальные входы syzkaller'а, который уже занимается монтированием образов, запуском действий с ними, сбором покрытия и падений.

Таким образом, общая схема работы представлена на Рис. 1

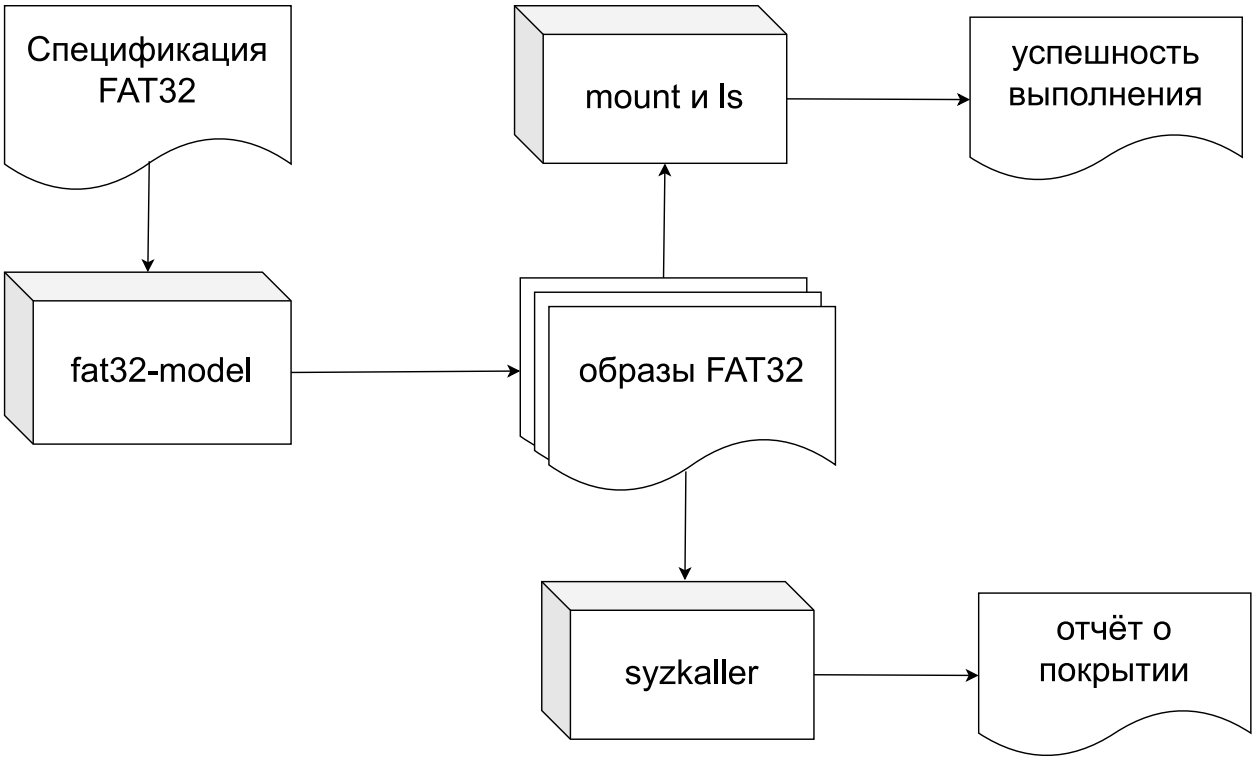

Рис. 1. Общая схема работы

4.1 Генерация образов

Специфицировать образ в бинарном виде напрямую в виде типа на Idris (то есть так, чтобы значениями типа были сразу нужные образы в бинарном виде и только они) на практике не представляется возможным, не говоря уже о том, что дериватор DepTyCheck не справится с типом такой сложности. Поэтому стандартным подходом является создание некоторого внутреннего представления (возможно даже нескольких, как иерархия моделей разных уровней абстракции), из которого уже будет собираться итоговый образ. Дополнительным преимуществом такого подхода будет

то, что мы сможем пользоваться этим внутренним представлением для определения корректности последующих системных вызовов, работающих с примонтированным образом ФС.

Файловая система FAT32 состоит из трёх основных частей:

1. Загрузочный сектор
2. FAT-таблица
3. Область данных

Область данных разбита на кластеры, в которые и записываются данные файлов и директорий. При этом кластеры, составляющие один файл, могут идти на диске не подряд, поэтому в FAT-таблице хранятся связные списки кластеров файлов. В загрузочном секторе хранится различная метаинформация о файловой системе.

Таким образом, генерацию можно разбить на несколько этапов, которые включают в себя генерацию дерева ФС, распределения файлов по кластерам, метаинформации и, наконец, сборку в итоговый бинарный образ.

Итоговый план генерации по этапам и зависимостям между ними представлен на Рис. 2, где `fuel` («топливо») — это параметр «глубины» генерации, принимаемый генераторами, сдеривированными DepTyCheck'ом.

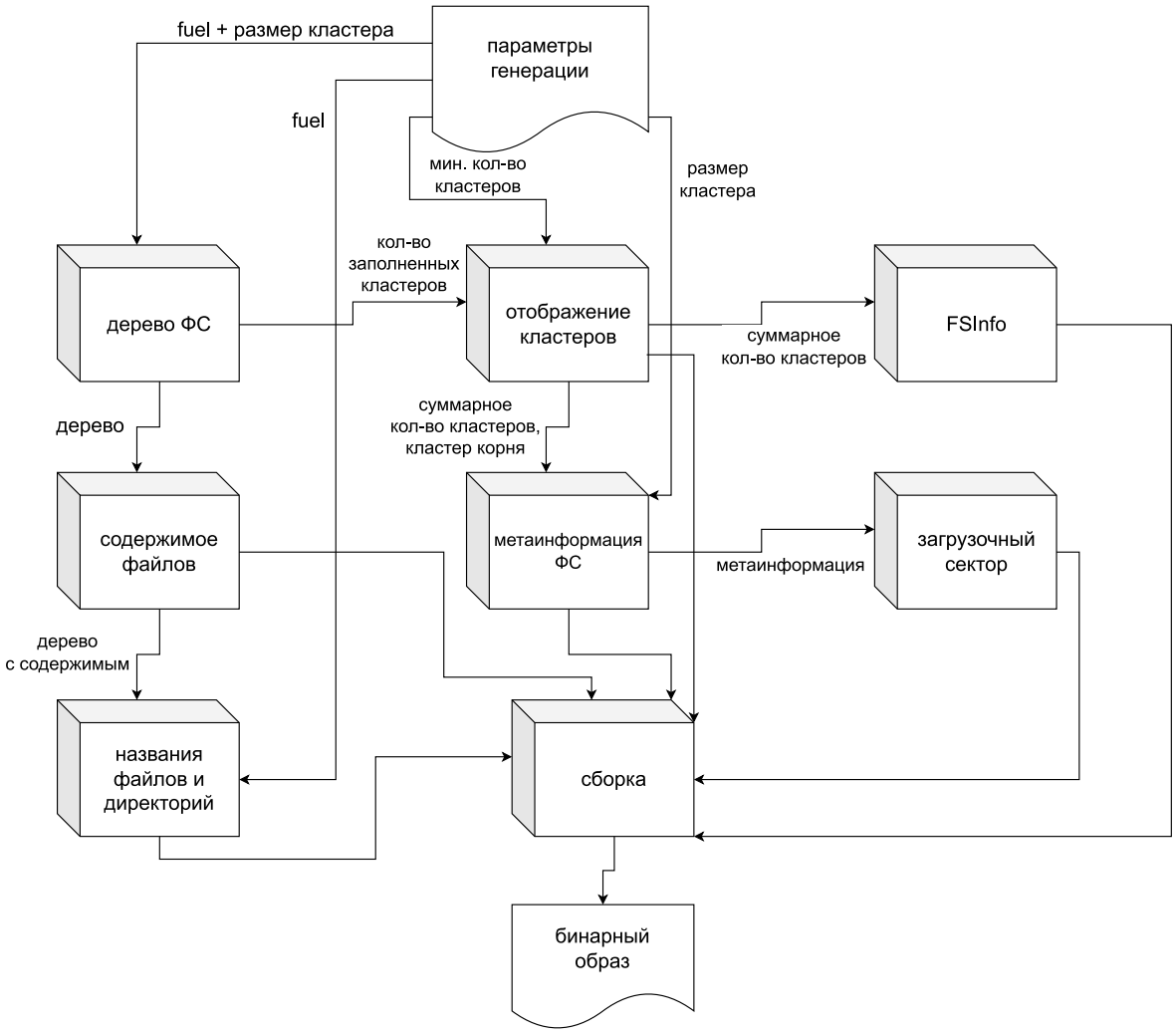

Рис. 2. Схема этапов генерации бинарного образа

4.1.1 Основная структура (дерево ФС)

На данном этапе генерируется дерево файлов и директорий, в котором содержится информация о флагах и размерах файлов (также в каждом узле дерева есть информация о том, сколько кластеров занимает этот файл/директория, а также суммарное количество занимаемых кластеров всего поддерева). При этом здесь ещё нет ни содержимого файлов, ни их названий. Такое разделение связано с особенностями работы DepTyCheck — если генерировать всё сразу, то больше «топлива» будет тратится на содержимое и имена вместо того, чтобы влиять на сложность структуры образа, которая является более желаемым свойством при создании сложных тестов. Кроме того, такое выделение логической структуры образа позволяет проще впоследствии добавить дополнительную функциональность, такую как генерация корректных последовательностей действий с файловой системой. Часть описания зависимого типа для этого этапа представлена на Листинге 1.

idris2

```
data Node : NodeCfg → NodeArgs → Bool → Bool → Type where
  FileB :
    (0 clustNZ : IsSucc clustSize) ⇒
    (meta : Metadata) →
    SnocVectBits8 k →
    Node (MkNodeCfg clustSize)
         (MkNodeArgs (divCeilNZ k clustSize) (divCeilNZ k clustSize) @{reflexive})
         True False
  Dir :
    (0 clustNZ : IsSucc clustSize) ⇒
    (meta : Metadata) →
    (entries : HSnocVectMaybeNode (MkNodeCfg clustSize) k ars wb False) →
    Node (MkNodeCfg clustSize) (
      let cur' = divCeilNZ (DirentSize * (2 + k)) clustSize
      in MkNodeArgs cur' (cur' + totsum ars) @{lteAddRight cur' {m = totsum ars}}
    ) wb False
```

Листинг 1. Часть описания дерева ФС

4.1.2 Содержимое файлов

Далее в дерево добавляется содержимое файлов. В дереве уже содержатся ожидаемые размеры файлов, на само содержимое нет практически никаких ограничений, контрольных сумм в FAT32 также нет, поэтому данный этап тривиально производится вручную без помощи DepTyCheck.

4.1.3 Имена файлов и директорий

В рамках данной работы не рассматривается расширение VFAT, добавляющее поддержку «длинных имён» (LFN), поэтому имена генерировались только в коротком формате 8.3, т.е. 8 байт на само имя, 3 байта на расширение (имя и расширение дополняются до нужных размеров пробелам) и ограниченный список поддерживаемых символов. Это несколько более сложная задача, чем генерация содержимого файлов, поскольку есть ограничения и на сами имена, и на то, что в одной директории они не должны повторяться. Таким образом, генератор самих имён был написан вручную, в то время как генератор дерева имён (с ограничением на уникальность в пределах директории) был сдеривирован DepTyCheck'ом (с использованием написанного генератора имён в качестве т.н. «внешнего генератора»). Часть описания типа дерева имён представлена на Листинге 2.

idris2

```
data NameTree : Node cfg ar wb fs → Type where
  FileB : {0 clustSize : Nat} →
          {0 clustNZ : IsSucc clustSize} →
          {0 k : Nat} →
          {0 blob : SnocVectBits8 k} →
          NameTree $ FileB @{clustNZ} meta blob {k}
  Dir : {0 clustSize : Nat} →
        {0 clustNZ : IsSucc clustSize} →
        {0 k : Nat} →
        {0 ars : SnocVectNodeArgs k} →
        {0 nodes : HSnocVectMaybeNode (MkNodeCfg clustSize) k ars True False} →
        UniqNames nodes →
        HSnocVectNameTree nodes →
        NameTree $ Dir @{clustNZ} meta nodes
```

Листинг 2. Часть описания дерева имён

4.1.4 Отображение виртуальных номеров кластеров в физические

Для создания образа кроме информации о самих файлах и директориях нужна также информация об их расположении на диске. Обычно при записи файлов на диск драйверы ФС по возможности заполняют свободные кластеры подряд, но поскольку наша задача состоит в том, чтобы потенциально получать любые возможные корректные образы, то нам нужно распределять файлы по кластерам случайным образом.

Первоначальный дизайн модели содержал информацию о кластерах каждого файла и директории уже в самом дереве ФС, но он получился слишком сложным, и с ним было проблематично работать (поскольку у структуры дерева появлялся новый инвариант — номера кластеров должны быть уникальными во всём дереве и находиться в определённом диапазоне). Поэтому модель дерева ФС была упрощена, а проблема распределения кластеров была решена другим способом. Информация о количестве занимаемых кластеров в узлах дерева позволяет пронумеровать все эти кластеры подряд при рекурсивном обходе дерева — это т.н. «виртуальные» номера. Тогда всё, что нужно, чтобы распределить кластеры по диску случайным образом, — это сгенерировать отображение «виртуальных» номеров на «физические», т.е. массив, размером с количество занятых кластеров, содержащий уникальные номера в диапазоне кластеров образа (по спецификации [7] номера кластеров начинаются с 2, а минимальное количество кластеров для FAT32 это 65525, при этом для управления «разреженностью» образа пользователь может увеличить минимальное количество кластеров в параметрах генерации).

Изначально этот генератор деривировался библиотекой, но позже был переписан вручную в целях оптимизации.

4.1.5 Метаинформация образа

Часть метаинформации образа нужна не только, чтобы записать в загрузочный сектор и забыть о ней, но и для последующей сборки образа, поэтому она генерируется (или извлекается из ранее полученных частей) отдельно. В частности, это такая информация, как итоговое количество кластеров, первый кластер корневой директории, количество FAT-таблиц и их размер, размер сектора, тип носителя и т.д.

4.1.6 Загрузочный сектор

Загрузочный сектор состоит из множества полей заданного размера, большинство из которых заполняются либо единственным образом, либо выбором из нескольких заданных вариантов, и чаще всего эти поля никак на дальнейшую генерацию не влияют (кроме тех, что были получены на предыдущем этапе). Так же, как и на предыдущем этапе, здесь используется рукописный генератор.

4.1.7 Блок FSInfo

Для оптимизации работы ФС в FAT32 есть ещё один служебный блок рядом с загрузочным сектором — FSInfo. Эта структура содержит по сути только два числа — количество свободных кластеров и номер следующего свободного кластера. При этом эта информация носит лишь справочный характер, поэтому для улучшения качества тестирования мы просто генерируем два случайных числа в нужном диапазоне.

4.1.8 Сборка бинарного образа

На данном этапе никакой генерации уже не происходит. Сначала копируются загрузочный сектор и FSInfo (дважды, т.к. у них должна быть ещё резервная копия), затем директории конвертируются в их бинарное представление (файлы уже в бинарном представлении, т.к. там просто случайное содержимое), файлы и директории бьются на кластеры и записываются в соответствии с отображением, соответствующие цепочки кластеров записываются в FAT-таблицу. После этого, если включено зеркалирование таблиц, то она ещё и копируется нужное число раз.

4.2 Оценка эффективности влияния на фаззеры

Для оценки эффективности метода полученные образы используются в качестве начальных данных для syzkaller'a. Для этого на протяжении нескольких дней было запущено 6 экземпляров фаззера, показанных в Таблице 1.

#	встроенные	сгенерированные	мутация образа
1	+		+
2		+	+
3	+	+	+
4	+		
5		+	
6	+	+	

Таблица 1. Запущенные экземпляры syzkaller'a

5 Результаты

5.1 Генератор

В результате проделанной работы был разработан генератор, исходный код которого доступен на GitHub [12]. Пример его работы продемонстрирован на Рис. 3 и Рис. 4. Параметры, которые можно задать генератору:

1. «зерно» (`seed`) генератора случайных чисел
2. «топливо» (`fuel`) генератора
3. размер кластера
4. минимальное количество кластеров
5. вывод внутреннего представления первых трёх этапов для отладки

Генерируемые образы соответствуют спецификации [7], но есть некоторые неподдерживаемые функции, а именно:

1. длинные имена (LFN)
2. генерация меток даты и времени создания/изменения (проставляются константой)
3. метка диска всегда «NO NAME»

```
~/Development/fat32-model
❯ pack run fat32-model -s 96769228 -c 512 -1 10 -2 20 -o "out_demo.img" -BT
generating Node...
generating NodeB...
Root {entries = :< (:< (:< (:< Lin (Just (Dir {meta = MkMetadata {readOnly = False, hidden = False, system = False, archive = True}, en
tries = :< (:< Lin Nothing) (Just (FileB (MkMetadata {readOnly = False, hidden = False, system = False, archive = False}) (:< Lin 169))
)}))) (Just (Dir {meta = MkMetadata {readOnly = True, hidden = False, system = True, archive = False}, entries = :< (:< (:< Lin Nothing
) (Just (FileB (MkMetadata {readOnly = False, hidden = False, system = False, archive = True}) (:< Lin 130)))) (Just (Dir {meta = MkMet
adata {readOnly = True, hidden = False, system = False, archive = True}, entries = :< Lin Nothing})})))) (Just (Dir {meta = MkMetadata
{readOnly = True, hidden = True, system = True, archive = True}, entries = :< (:< Lin Nothing) (Just (FileB (MkMetadata {readOnly = Fal
se, hidden = True, system = False, archive = True}) (:< (:< (:< (:< (:< (:< (:< (:< (:< (:< Lin 155) 235) 184) 197) 59) 152) 135) 31) 1
55) 142)))})))) (Just (Dir {meta = MkMetadata {readOnly = False, hidden = False, system = False, archive = True}, entries = :< Lin (Just
 (Dir {meta = MkMetadata {readOnly = False, hidden = True, system = True, archive = False}, entries = :< Lin Nothing})})))}
generating NameTree...
Root (NewName {f = MkFilename (:: 96 (:: 72 (:: 53 (:: 32 (:: 32 (:: 32 (:: 32 (:: 32 (:: 126 (:: 79 (:: 123 Nil)))))))))))}) (:< (:< (
:< (:< Lin (Just (Dir (NewName {f = MkFilename (:: 85 (:: 35 (:: 125 (:: 51 (:: 32 (:: 32 (:: 32 (:: 32 (:: 56 (:: 76 (:: 32 Nil)))))))
))))}) (:< (:< Lin Nothing) (Just FileB))))) (Just (Dir (NewName {f = MkFilename (:: 82 (:: 37 (:: 80 (:: 74 (:: 73 (:: 32 (:: 32 (:: 3
2 (:: 88 (:: 56 (:: 32 Nil)))))))))))}) (:< (:< (:< Lin Nothing) (Just FileB)) (Just (Dir (NewName {f = MkFilename (:: 89 (:: 71 (:: 32
 (:: 32 (:: 32 (:: 32 (:: 32 (:: 32 (:: 48 (:: 77 (:: 69 Nil)))))))))))}) (:< Lin Nothing))))))) (Just (Dir (NewName {f = MkFilename (:
: 88 (:: 86 (:: 88 (:: 32 (:: 32 (:: 32 (:: 32 (:: 32 (:: 86 (:: 32 (:: 32 Nil)))))))))))}) (:< (:< Lin Nothing) (Just FileB))))) (Just
 (Dir (NewName {f = MkFilename (:: 68 (:: 79 (:: 67 (:: 89 (:: 73 (:: 90 (:: 78 (:: 32 (:: 65 (:: 40 (:: 36 Nil)))))))))))}) (:< Lin (J
ust (Dir (NewName {f = MkFilename (:: 84 (:: 32 (:: 32 (:: 32 (:: 32 (:: 32 (:: 32 (:: 32 (:: 65 (:: 32 (:: 32 Nil)))))))))))}) (:< Lin
 Nothing)))))))
generating cmap...
generating bdata...
generating bsect...
generating fsinfo...
building the image...
~/Development/fat32-model  on  main                                  0.55  90%  20.32G  at 12:02:57
❯
```

Рис. 3. Демонстрация генерации образа с выводом внутренних представлений

```
❯ df -h | grep loop0
/dev/loop0          32M        294K   32M          1% /home/elk/mnt
❯ mount | grep loop0
/dev/loop0 on /home/elk/mnt type vfat (rw,relatime,fmask=0022,dmask=0022,codepage=4
37,iocharset=ascii,shortname=mixed,utf8,errors=remount-ro)
❯ lsd -l --tree --date + ~/mnt
drwxr-xr-x root root 1.0 KB  mnt
drwxr-xr-x root root 512 B   ├── %WB_.&RO
drwxr-xr-x root root 512 B   │   ├── #`)B0G.L
drwxr-xr-x root root 512 B   │   │   ├── "'!$U(T%G.)3B"
drwxr-xr-x root root 512 B   │   │   ├── 'A-$}XC92.{KL'
.r-xr-xr-x root root   9 B   │   │   │   ├── @J6.JH
drwxr-xr-x root root 512 B   │   │   │   ├── AHR3~5.U(
.rwxr-xr-x root root   3 B   │   │   │   │   └── "Z^.M'"
drwxr-xr-x root root 512 B   │   │   │   ├── JXKF6ARJ.`MI
drwxr-xr-x root root 512 B   │   │   │   ├── 'PG&B8$.X'
.r-xr-xr-x root root  12 B   │   │   │   └── {GJ8LP^.70B
drwxr-xr-x root root 512 B   │   │   ├── E}W25F7.8
drwxr-xr-x root root 512 B   │   │   ├── RD^}AL.8G
drwxr-xr-x root root 512 B   │   │   ├── SZRSTD(.~
.rwxr-xr-x root root  11 B   │   │   └── U._2
drwxr-xr-x root root 512 B   │   ├── (0SDYV.EG8
drwxr-xr-x root root 512 B   │   │   ├── 9%W9Y1.XUD
drwxr-xr-x root root 512 B   │   │   │   ├── "(XS.'G"
drwxr-xr-x root root 512 B   │   │   │   ├── 5D5).#A
drwxr-xr-x root root 512 B   │   │   │   │   ├── `_7)5IV.Z
drwxr-xr-x root root 512 B   │   │   │   │   └── SQ(@G%.S9
.r-xr-xr-x root root  22 B   │   │   │   ├── 6.XC)
.rwxr-xr-x root root   4 B   │   │   │   ├── BXO.0E
.r-xr-xr-x root root  14 B   │   │   │   ├── C17.Z
drwxr-xr-x root root 512 B   │   │   │   └── 'P$N@L_.F5C'
drwxr-xr-x root root 512 B   │   │   ├── `B3C6-{F.3Q
drwxr-xr-x root root 512 B   │   │   │   ├── &HYUC#I.!N5
.r-xr-xr-x root root   0 B   │   │   │   │   ├── (5B.Q
.r-xr-xr-x root root   7 B   │   │   │   │   ├── B)W2~Y7P.H22
drwxr-xr-x root root 512 B   │   │   │   │   ├── WSJ.SYT
drwxr-xr-x root root 512 B   │   │   │   │   ├── YB6W9K.^
drwxr-xr-x root root 512 B   │   │   │   │   │   ├── 7.`Z
.r-xr-xr-x root root  21 B   │   │   │   │   │   └── `Y_PEFV.O
drwxr-xr-x root root 512 B   │   │   │   │   └── "~'IK~U@.22"
.r-xr-xr-x root root  13 B   │   │   │   ├── 'L$}05.JE_'
drwxr-xr-x root root 512 B   │   │   │   └── T%NQP.U%
.r-xr-xr-x root root   6 B   │   │   │       ├── #.}K0
drwxr-xr-x root root 512 B   │   │   │       ├── "'!_QQ#.23E"
```

Рис. 4. Результат генерации

5.2 Несоответствие спецификации

В процессе тестирования было обнаружено, что некоторые образы не монтировались сразу на нескольких популярных операционных системах (Windows 10, Linux 6.14, macOS Sequoia). Выяснилось, что ни одна из перечисленных систем не поддерживает функцию отключения зеркалирования FAT-таблиц, при этом не сигнализируя об этом должным образом. Windows не обнаруживает ФС FAT32 на диске, если флаг зеркалирования выключен, в то время как Linux и macOS всегда смотрят только

в нулевую таблицу (см. код [13]), независимо от значения флага, приводя к таким ошибкам, как на Листинге 3.

```
kernel: FAT-fs (loop0): error, fat_get_cluster: invalid cluster chain (i_pos 1)
kernel: FAT-fs (loop0): Filesystem has been set read-only
```

Листинг 3. Ошибка, выводимая драйвером FAT32 на Linux 6.14 при попытке примонтировать образ с выключенным зеркалированием и активной таблицей №1

5.3 Влияние на работу фаззера

5.3.1 Сложности работы с syzkaller'ом

Использование syzkaller'а для оценки эффективности метода оказалось довольно проблематичным по нескольким причинам. Во-первых, в процессе фаззинга довольно много решений происходит с элементами случайности, контролировать которую не представляется возможным. Это усложняет сравнение работы фаззеров с разными конфигурациями, поскольку приходится учитывать фактор случайности. От этого же страдает и воспроизводимость экспериментальной установки. Во-вторых, syzkaller сложно сфокусировать на фаззинге FAT32 или даже просто на фаззинге файловых систем — есть только возможность ограничивать генерируемые системные вызовы. Незадолго до исследования появилась экспериментальная функция, позволяющая выставлять приоритет определённым частям кодовой базы, но на практике она работала очень нестабильно. В-третьих, механизм сопоставления собранного покрытия конкретным частям кода также очень нестабилен, и статус инструментированности некоторых строк мог разниться даже между перезапусками.

Наконец, оказалось, что в коде распаковки начальных образов есть ограничение на размер — распакованный образ должен быть не больше 132 МиБ. При этом образ FAT32 минимального размера с кластерами по 4 КиБ уже превышает это ограничение, поэтому пришлось увеличить константу в коде до 300 МиБ.

5.3.2 Анализ покрытия

Анализ производился по «сырому» покрытию, представляющему из себя множество адресов покрытых базовых блоков. Для каждой пары фаззеров внутри одной группы (с мутацией образов и без неё соответственно) сравнивались количества уникальных адресов. На Рис. 5 представлен график, где каждая двухцветная полоска соответствует паре фаззеров. На пометках в скобках указано количество адресов, которые были у обоих фаззеров в паре, а выше — количество адресов, уникальных для этого фаззера, и сколько процентов это составляет от пересечения.

Во-первых, в каждой паре получилось очень большое пересечение в покрытии, так что обсуждаемые далее превосходства имеют маргинальный характер. Во-вторых, из графика видно, что фаззер на чисто сгенерированных `fat32-model` образах показал результат лучше, чем остальные (на фаззерах без мутаций похожая картина, но в меньших масштабах). Довольно необычно ведёт себя фаззер, работавший на образах обоих видов. Возможно, на это повлиял тот факт, что у него было в два раза больше начальных образов, чем у остальных, и было меньше времени на более «глубокое» использование каждого.

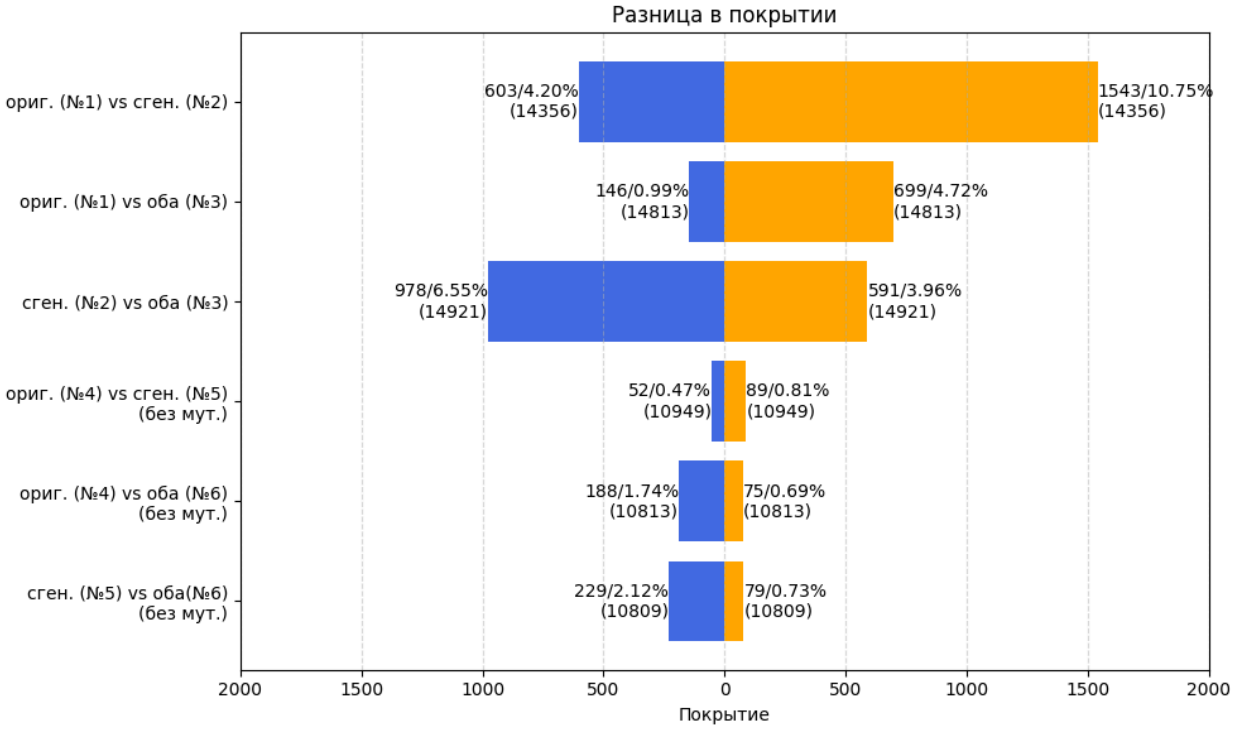

Рис. 5. График сравнения покрытия различных фаззеров

6 Заключение

В данной работе была создана формальная спецификация для семантически корректных образов файловой системы FAT32 с использованием зависимых типов и на основе неё был разработан генератор этих образов. С помощью данных образов были протестированы драйверы трёх операционных систем (Windows, Linux, macOS) и были выявлены несоответствия спецификации во всех трёх системах. С помощью syzkaller был проведён анализ покрытия кода Linux при фаззинге со сгенерированными разработанным инструментом образами в качестве начальных и обнаружено небольшое их преимущество по сравнению с образами, вшитыми в syzkaller по умолчанию. Сделан вывод о проблематичности использования syzkaller'а для исследований подобного рода.

Возможные направления для дальнейших исследований включают в себя использование других методов оценки эффективности тестирования сгенерированными образами, улучшение генератора путём добавления поддерживаемых функций FAT32 (таких, как длинные имена и метки времени), а также разработка генератора последовательностей семантически корректных действий с файловой системой для дальнейшего функционального тестирования драйверов.

Список литературы

[1] E. Brady, «Idris, a general-purpose dependently typed programming language: Design and implementation», *Journal of Functional Programming*, т. 23, вып. 5, сс. 552–593, 2013, doi: 10.1017/S095679681300018X.

[2] E. Brady, «Idris 2: Quantitative Type Theory in Practice», 2021 г., 2104.0048. [Онлайн]. Доступно на: https://arxiv.org/abs/2104.00480

[3] «GitHub-репозиторий Idris 2». [Онлайн]. Доступно на: https://github.com/idris-lang/Idris2

[4] D. Buzdalov, «Dependent Types to Push Corners of the Property-based Testing (Extended Abstract)», 6 сентябрь 2024 г. [Онлайн]. Доступно на: https://icfp24.sigplan.org/details/tyde-2024-papers/6/Dependent-Types-to-Push-Corners-of-the-Property-based-Testing-Extended-Abstract-

[5] «GitHub-репозиторий DepTyCheck». [Онлайн]. Доступно на: https://github.com/buzden/deptycheck

[6] D. Christiansen и E. Brady, «Elaborator reflection: extending Idris in Idris», в *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, Association for Computing Machinery, 2016, сс. 284–197. doi: 10.1145/2951913.2951932.

[7] «Microsoft Extensible Firmware Initiative FAT32 File System Specification, FAT: General Overview of On-Disk Format». [Онлайн]. Доступно на: https://msdn.microsoft.com/en-us/windows/hardware/gg463080.aspx

[8] «GitHub-репозиторий syzkaller». [Онлайн]. Доступно на: https://github.com/google/syzkaller

[9] «Генерация начальных образов в коде syzkaller». [Онлайн]. Доступно на: https://github.com/google/syzkaller/blob/master/tools/syz-imagegen/imagegen.go

[10] W. Xu, H. Moon, S. Kashyap, P.-N. Tseng, и T. Kim, «Fuzzing File Systems via Two-Dimensional Input Space Exploration», в *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, сс. 818–834. doi: 10.1109/SP.2019.00035.

[11] M. P. Mehta, «Formalising Filesystems in the ACL2 Theorem Prover: an Application to FAT32», *Electronic Proceedings in Theoretical Computer Science*, т. 280, сс. 18–29, окт. 2018, doi: 10.4204/eptcs.280.2.

[12] «GitHub-репозиторий fat32-model». [Онлайн]. Доступно на: https://github.com/DepTyCheck/fat32-model

[13] «Elixir Cross Referencer, fs/fat/inode.c, строка 1651». [Онлайн]. Доступно на: https://elixir.bootlin.com/linux/v6.14.9/source/fs/fat/inode.c#L1651